

开源软件的演化：一项案例研究

迈克尔·W·戈弗雷和涂强
滑铁卢大学计算机科学系软件架构小组 (SWAG)

电子邮件: {migod,qtu}@swag.uwaterloo.ca

摘要

大多数关于软件演化的研究都是针对使用传统管理技术在单一公司内部开发的系统进行的。随着一些采用“开源”开发方式开发的大型软件系统的广泛可用,我们现在有机会详细研究这些系统,看看它们的演化历程是否与商业开发的系统有显著差异。本文总结了我们对最著名的开源系统——Linux 操作系统内核演化的初步研究。由于 Linux 规模庞大(最新版本有超过两百万行代码),且其开发模式不像大多数工业软件流程那样经过严密规划和管理,我们原本预计随着 Linux 变得更大、更复杂,其增长速度会变慢。然而,我们发现 Linux 多年来一直以超线性速率增长。在本文中,我们从系统层面和主要子系统内部探讨了 Linux 内核的演化,并讨论了我们认为 Linux 持续呈现如此强劲增长态势的原因。

1 引言

大型软件系统必须不断演进,否则就有可能将市场份额拱手让给竞争对手 [151]。然而,维护这样一个系统极其困难、复杂且耗时。随着系统的老化和规模扩大,添加新功能、增加对新硬件设备和平台的支持、系统调优以及修复缺陷等任务都变得愈发困难。

大多数已发表的关于软件演化的研究都是针对在单一公司内部使用传统开发和管理技术“自行开发”的系统进行的 [3, 4, 12, 15, 23]。在本文中,我们对 Linux 操作系统的演化进行了案例研究。

[9, 8, 71. 本系统采用“开源开发”方法进行开发,这与大多数工业软件的创建方式截然不同 [18]。

2 相关工作

雷曼等人针对大型、长生命周期软件系统的演化开展了规模最大且最为知名的研究 [13, 15, 14, 23]。雷曼基于对多个大型软件系统的案例研究,提出了软件演化定律 [15]。该定律表明,随着系统规模的增长,除非采取明确措施对整体设计进行重组,否则添加新代码的难度会越来越大。图尔斯基对这些案例研究进行的统计分析表明,系统增长(以源模块数量和变更模块数量衡量)通常呈次线性,随着系统规模和复杂度的增加而放缓 [14, 23]。

高尔等人从系统层面以及顶级子系统层面,对一个大型电信交换系统的演化进行了研究 [4],这与我们对 Linux 所做的研究颇为相似。他们指出,尽管该系统在系统层面的演化似乎符合传统观察到的随时间推移变化率降低的趋势,但主要子系统的表现可能与整个系统有很大不同。在他们的案例研究中,他们发现一些主要子系统呈现出“有趣的”演化行为,但从顶层审视整个系统时,这些行为相互抵消了。因此,他们认为仅从最高层面考虑演化是不够的,还必须关注各个组成部分。我们自己的研究也有力地支持了这一观点。

凯默勒 (Kemerer) 和斯劳特 (Slaughter) 对软件演化方面的研究进行了出色的综述 [12]。他们还指出,关于软件演化实证研究的相关成果相对较少。

帕纳斯用“衰退”这一比喻来描述如何

以及为什么软件会随着时间的推移而变得越来越脆弱 [16]。艾克等人扩展了帕纳斯提出的观点，以可检测和测量的方式描述软件“退化” [3]。他们以一个大型电话交换系统为例进行研究。例如，他们指出，如果修复缺陷通常需要对大量源文件进行修改，那么该软件系统的设计可能很糟糕。他们的衡量标准基于详细的缺陷跟踪日志的可用性，例如，用户可以通过这些日志确定有多少缺陷导致了特定模块的修改。我们注意到，在我们对 Linux 的研究中，没有这样详细的变更日志。

佩里提供的证据表明，软件系统的演化不仅取决于其规模和使用时长，还取决于诸如系统本身的性质（即其应用领域）、之前使用该系统的经验，以及开发该软件的公司所采用的流程、技术和组织架构等因素 [17]。

3 开源软件开发

尽管“开源”这一术语相对较新，但其背后的基本理念并非如此 [10, 181]。开源软件系统最重要的一个要求是，其源代码必须免费提供给任何希望查看或出于自身目的修改它的人。也就是说，用户必须始终能够“查看系统内部”，并被允许根据个人需求对系统进行调整、适配或改进。

虽然开源软件（OSS）的开发通常具有高度的协作性，且在地理上分布广泛，但这并非严格要求。许多公司和个人将源代码作为专有项目在内部开发，之后才将其作为“开源”发布，或者采用允许系统个人使用有很大自由度的许可进行发布。这方面的例子包括网页浏览器（即 Mozilla 项目）、IBM 的 Jikes Java 编译器，以及 Sun 公司的 Java 开发工具包。

另一种开源系统是从早期就作为一个高度协作的“公开”项目进行开发的；也就是说，这些系统遵循开源开发（OSD）模式。通常，这样的项目由单个开发者发起，他们有着个人目标或愿景。一般来说，这个人会从零开始开发系统，或者借鉴现有的旧系统。例如，Linux 的创造者莱纳斯·托瓦兹（Linus Torvalds）从一个版本的 Minix 操作系统起步，而 VIM 的创造者布拉姆·莫伦纳尔（Bram Moolenaar）则从一个名为

“埃里克·雷蒙德（Eric Raymond）写了一本关于开源开发的内容丰富的书，名为《大教堂与集市》[18]。”

3.1 开源开发与传统流程

一旦发起者准备好邀请他人参与项目，他/她就会将代码库提供给其他人，开发工作便由此展开。通常，任何人都可以为系统的开发做出贡献，但发起者/所有者有权决定哪些贡献可以或不可以成为官方版本的一部分。2

开源开发（OSD）模式在几个基本方面与传统的内部商业开发流程不同。首先，开源项目的通常目标是创建一个对参与其中的人有用或有趣的系统，而不是填补商业空白。开发者通常是无偿的志愿者，他们将参与项目作为一种爱好；作为回报，他们获得同行的认可以及他们的努力所带来的个人满足感。有时这意味着，开源开发项目的大部分精力都集中在兼职程序员感兴趣的方面，而不是那些可能更关键的方面。由于项目所有者对参与开发的人员几乎没有控制权，因此可能很难将开发引导向特定目标。这种自由也意味着，可能很难说服开发者去执行诸如系统测试或代码重构等必要任务，因为这些任务不像编写新代码那样令人兴奋。

开源系统的测试

开源开发的其他显著特点包括：

- 时间安排：通常没有太大的商业压力要求严格遵守时间表，而且大多数开源软件开发者都有“日常工作”，占据了他们大部分时间。虽然这可能意味着开发周期较长，但这也是一个优势，因为开源软件项目在很大程度上不受“上市时间”压力的影响；在项目所有者对系统的成熟度和稳定性感到满意之前，无需发布系统。

代码质量和标准可能差异很大。由于代码是由多方贡献的，很难坚持特定的标准，尽管许多项目确实有官方指南。

不稳定的代码很常见，因为开发人员急于将他们的“前沿”贡献提交到项目中。一些开源开发（OSD）项目，包括 Linux，通过维护两条并行的开发路径来解决这个问题：“开发版”发布路径包含新功能或实验性功能，而“稳定版”发布则包含

“一些开源项目在开发者对‘官方’分支所采取的路线不满意时，已经分叉成不同的开发流。在大多数开源许可协议下，这种分叉是明确允许的。FreeBSD/NetBSD/OpenBSD 系统就是这种现象的一个例子。”

主要是相对于上一个稳定版本的更新和错误修复。此外，当新的“开发中”功能被认为已稳定时，有时会将它们迁移到当前稳定版本中，而无需等待建立下一个主要基线。）

@ 有计划的演进、测试和预防性维护。

这可能会影响代码质量，因为开源软件开发鼓励积极参与，但不一定鼓励认真思考和重新组织。代码质量主要通过“大规模并行调试”即许多开发人员相互使用彼此的代码来维持，而非通过系统测试或其他有计划的、规范性的方法。3

3.2 开源软件开发系统的演进

我们研究了开源软件开发（OSD）项目的增长和演化模式，以了解它们与之前关于使用更传统的内部流程开发的大型专有软件系统演化的研究相比情况如何。现在有许多大型开源软件开发系统已经存在多年并得到广泛应用，其中包括我们详细研究过的两个系统：Linux 操作系统内核（220 万行代码）[9]，以及 VIM 文本编辑器（15 万行代码）[11]。

天真地讲，我们曾预期，由于开源软件开发的演进通常远不如传统内部开发那样有条理和精心规划，因此“莱曼定律”会适用[15]；也就是说，随着系统的增长，增长速度会放缓，系统增长近似于一条平方反比曲线[14]。事实上，Perl 项目的维护者最近对核心系统进行了大规模的重新设计和架构调整[20]，因为项目所有者觉得当前系统几乎已变得难以维护。然而，正如我们在下面所解释的，我们在 Linux 上发现的情况并非如此。

Perl 和 Linux 还真不一定

FreeBSD 操作系统是 Linux 的竞争对手，其开发模式[6]介于传统的严格管理开发模式与许多开源开发（OSD）项目采用的相对无组织的方法之间。FreeBSD 系统接受外部人员的贡献，但在将这些贡献纳入主源代码树之前，会进行更仔细的审查。FreeBSD 开发团队对代码的测试也比 Linux 严格得多。因此，与 Linux 相比，FreeBSD 支持的设备往往较少，开发进展也较慢。

对 VIM 的 4.0~r 初步分析表明，多年来它也一直在以超线性速度增长。

核心开发者之一奇普·萨尔岑伯格（Chip Salzenberg）这样评价当前版本的 Perl：“在真正期望对 Perl 内核做出重大修改且不造成得不偿失的破坏之前，你确实需要深入了解所有的奥秘和神奇结构等。”[20]

4 Linux 操作系统内核

Linux 是一种类 Unix 操作系统，最初由莱纳斯·托瓦兹编写，随后有数百名其他开发者参与其中。它最初是为在英特尔 386 架构上运行而编写的，但后来已被移植到许多其他平台，包括 PowerPC、DEC Alpha、Sun SPARC 和 SPARC64，甚至大型机和个人数字助理（PDA）。

内核的第一个正式版本 1.0 于 1994 年 3 月发布。这个版本包含 487 个源代码文件，代码行数超过 165,000 行（包括注释行和空行）。从那时起，Linux 内核沿着两条并行的路径进行维护：一个开发版本，包含实验性且相对未经测试的代码；以及一个稳定版本，主要包含相对于上一个稳定版本的更新和错误修复。按照惯例，内核版本号中的中间数字标识其所属路径：奇数（例如 1.3.49）表示开发内核，偶数（例如 2.0.7）表示稳定内核。

在撰写本文时（2000 年 1 月），最新的稳定内核版本是 2.2.14，而最新的开发内核版本是 2.3.39。沿着四条主线（1.1.x、1.3.x、2.1.x 和 2.3.x）已经发布了 369 个开发版本，沿着四条主线（1.0.x、1.2.x、2.0.x 和 2.2.x）已经发布了 67 个稳定内核版本。

5 方法论

我们使用了多种工具和假设，对 Linux 发展的各个方面进行了衡量，下面将对此展开描述。

我们总共研究了 96 个内核版本，其中包括 34 个稳定内核版本和 62 个开发内核版本。由于稳定内核发布频率较低，我们决定相对更多地测试它们。

完整发行版的大小是通过使用 gzip 压缩的 tar 文件来衡量的；此文件包含内核的所有源文件，包括文档、脚本和其他实用程序（但不包含二进制文件）。也就是说，这些 tar 文件是可从 Linux 内核归档网站[8]获取的版本。

我们使用“源文件”一词来指代原始 tar 文件中出现的、文件名以“.c”或“.h”结尾的任何文件。我们忽略了其他源文件，如配置文件和 Makefile。我们还明确忽略了

2000 年 1 月发布的内核版本 2.3.39 在致谢文件中列出了 300 多个对 Linux 内核开发做出重大贡献的人员名字。

执行系统构建会根据所选项创建额外的源文件。为了保持一致性和简洁性，我们忽略了这些额外的文件。

那些出现在“Documentation”目录下的源文件，因为我们认为这些本身并非内核代码的一部分。

我们使用两种方法统计代码行数（LOC）：第一，我们使用 Unix 命令“WC -l”，该命令给出的原始统计结果中，总行数包含了空行和注释；第二，我们使用一个 awk 脚本，该脚本会忽略空行和注释。最后，我们使用程序“exuberant ctags”来统计全局函数、变量和宏的数量 [5]。

在考虑 Linux 的主要子系统时，我们使用每个源代码版本的目录结构来定义子系统层次结构。其他人则基于对特定版本 Linux 的详细分析，创建了定制的子系统层次结构（即基于源代码的软件架构）[2, 22, 21]。我们选择不采用这种方法有两个原因：第一，为 96 个版本的 Linux 内核创建定制的子系统层次结构是一项艰巨的任务，且每个内核包含 500 到 5000 个源文件，却没有明显的好处；第二，我们对子系统演变分析将取决于我们自己对软件架构应有的样子的想法，而不符合大多数 Linux 开发者的思维模式。

雷曼建议使用“模块”数量作为衡量大型软件系统规模的最佳方式 [151]。然而，出于几个原因，我们决定在大多数测量中使用无注释代码行数（“无注释代码行”）。首先，如下所述，我们发现系统的总无注释代码行似乎与源文件数量的增长速度大致相同；然而，正如下面平均文件大小和中位数文件大小之间的差异所示，系统某些部分的文件大小存在很大差异。因此，我们决定，使用源文件数量意味着会遗漏 Linux 演化的一些完整情况，尤其是在子系统层面。

6 关于 Linux 演化的观察

我们在系统层面以及每个主要子系统内部研究了 Linux 内核的演变；我们发现，整个系统呈现出如此强劲的增长速度，因此对主要子系统进行调查是恰当的。现在我们来讨论我们的观察结果。

我们还绘制了这两个计数之间的差异，发现源文件中注释行或空行的百分比几乎保持在 28% 到 30% 之间。我们认为这种稳定性是一个好迹象。注释量的减少通常表明代码维护不善，而注释量的显著增加通常表明系统变得难以理解，需要额外的解释。

6.1 系统层面的增长

我们首先使用几种常见指标研究了该系统是如何发展的。例如，图 1 展示了完整内核版本的压缩 tar 文件大小的增长情况，图 2 展示了代码行数（LOC）的增长情况。我们还测量了源文件数量的增长以及全局函数、变量和宏数量的增长；然而，为简洁起见，我们省略了这些测量的图表，因为它们所显示的增长模式与图 1 和图 2 的非常相似。

有趣的是，这些测量结果似乎都说明了同一个问题。它们清楚地表明，开发版本随时间以超线性速率增长，这与莱曼

（Lehman）和图尔斯基（Turski）的平方反比增长率假设 [14, 23] 相悖。9 早期的稳定内核路径（版本 1.2.X 和 2.0.X，分别从 1995 年 3 月和 1996 年 7 月开始，可在图表中看到）的增长速度比相应的开发版本路径慢得多，这与预期相符。然而，最新的稳定版本路径，即从 1999 年 1 月开始的 2.2.X 版本，作为一条“稳定”路径却展现出了显著的增长。后续调查发现，这种增长主要来自新功能的添加和对新架构的支持，而非缺陷修复 [7]。最近 Linux 的迅速流行，使得大量来自第三方的稳定代码得以贡献，比如 IBM 为其 S/390 大型机贡献的代码 [24]，同时也面临着将这些代码“快速整合”到稳定版本路径中的外部压力。

我们将所有增长情况与时间而非版本号进行了绘图（Lehman 等人建议采用后者 [14]）。对我们来说，将在实际时间中并行的稳定版本和开发版本路径按顺序绘制是没有意义的；这样做会导致在一种发布路径结束而另一种开始的地方出现明显的“下降”。此外，这两种路径的表现有所不同是可以理解的：开发内核发布频率很高，且大小差异很大，而早期稳定内核发布的频率各不相同，但（在 2.2.X 版本之前）大小相对较小。我们还注意到，Linux 显然并不遵循 Lehman 的软件演化第三定律，该定律指出，在系统的整个生命周期中，每次发布所投入的增量工作量保持不变 [15]；例如，2.3.X 发布路径中的更新“补丁”文件

“我们的统计分析表明，在开发发布路径上，无注释代码行（LOC）的增长率与二次模型拟合得很好。如果 x 是自 1.0 版本发布以来的天数， r 是 Linux 内核无注释代码行的规模，那么使用“最小二乘法”近似计算得出，以下函数是增长的良好模型：

$$Y = 0.21 * X^2 + 252 * X + 90,055$$

该模型的决定系数为 0.997。

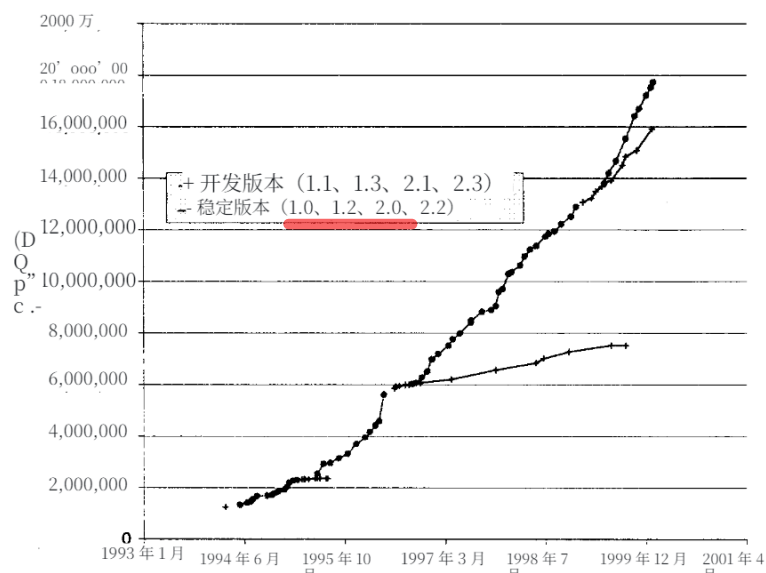


图 1. 完整 Linux 内核源代码发行版的压缩 tar 文件的增长情况。

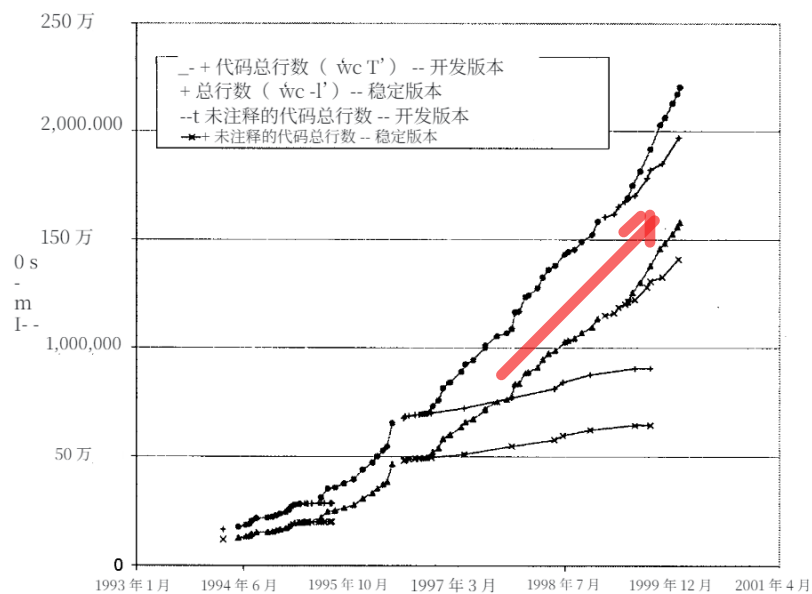


图 2. 使用两种方法测量的代码行数增长情况：Unix 命令 “WC -l”，以及一个用于去除注释和空白行的 awk 脚本。

大小从几百字节到几兆字节不等。基于上述原因，我们假设这是 OSD 流程模型的直接结果。

由于我们所使用的各项指标的增长模式似乎相似，因此我们尝试将一种度量值除以另一种度量值，看看所得曲线是否为直线。于是，我们绘制了平均文件大小（未注释的总行数除以源文件数量），以及“.c”实现和“.h”头源文件的中位数大小。图 3 和图 4 表明，中位数文件大小相当稳定，.c 文件大小略有增长，而平均文件大小随时间有明显增长。这表明，虽然一些.c 文件变得相当大，但大多数并非如此。我们随后的调查显示，几乎所有最大的文件都是复杂硬件设备的驱动程序。我们认为.h 文件大小中位数曲线的平稳性是个好迹象，因为这表明新功能并非随意添加到.h 文件中。对.h 文件平均大小上升的调查显示，有少量非常大的设备驱动程序.h 文件，其中大多包含数据；这些非常大的.h 文件往往会使整体平均文件大小产生偏差。

从 1996 年年中开始出现了一个有趣的趋势，即稳定版本 2.0.X 及其并行开发版本 2.1.X，如图 3 和图 4 所示。沿着 2.0.X 版本路径，我们可以看到平均 dot-c 文件大小有所增加，而沿着 2.1.X 版本路径，该值在最终再次开始增加之前有所下降。同时，沿着 2.0.X 版本路径，平均 dot-h 文件大小缓慢但稳定地增加，而沿着 2.1.X 版本路径，平均 dot-h 文件大小增加得更为显著，并主导了稳定版本路径。通过将此图与文件数量的增长进行交叉参考，我们注意到，沿着稳定版本 2.0.X，源文件的数量增长缓慢，而沿着 2.1.X，源文件的数量有较大幅度的稳定增长，在其平均 dot-c 文件大小下降的同一时间点出现了显著跃升。”这表明，沿着稳定版本路径，平均文件大小的增加很可能是由于错误修复和简单增强，这些操作向现有文件中添加了代码，这与人们对稳定版本路径的预期一致。沿着开发版本路径，很可能由于添加了新功能而创建了许多新的小文件，导致平均 dot-c 文件大小下降。我们认为新开发……

“例如，在最新的开发内核（v2.3.39）中，8 个最大的.c 文件中有 6 个是 SCSI 卡驱动程序，而 6 个最大的.h 文件中有 4 个是网卡驱动程序。这让我们初步推测，SCSI 卡的逻辑复杂，而网卡的接口复杂。结果发现，虽然 SCSI 卡确实逻辑复杂，但网卡.h 文件的大部分内容只是数据。”

1. 同时，在两条路径上，.c 文件与 .h 文件数量的比例保持恒定。

“1997 年 1 月发布 2.1.20 版本时，dot-h 文件的平均大小大幅增加，这主要是因为添加了一个非常大的网卡设备驱动 dot-h 文件。”

这种安排似乎产生了额外的基础设施（即新的小文件），这些基础设施随后会随着时间的推移“完善”。

6.2 主要子系统的增长

在对 Linux 在系统层面的发展情况进行调研后，我们决定进一步研究主要子系统（按照源目录层次结构定义）的发展情况。主要有十个源子系统 [19]：

- drivers 包含大量针对各种硬件设备的驱动程序；

0 架构包含特定于特定硬件架构（如 KPU）的内核代码，包括对内存管理和库的支持；

- include 目录包含了系统的大部分头文件（.h 文件）；

0 网络包含主要的网络代码，如对套接字和 TCP/IP 的支持（特定网卡的代码包含在 drivers/net 子系统中）；

fs 包含对各种文件系统的支持；

- init 包含内核的初始化代码；

- ipc 包含进程间通信的代码；

内核包含与硬件架构无关的主要内核代码；

体系结构无关；

- lib 包含（与架构无关的）库代码；并且

- mm 包含（与架构无关的）内存管理代码。

图 5 展示了主要内核子系统的增长情况。我们可以立刻看到，驱动程序子系统不仅是最大的子系统，也是增长最快的子系统，在最新版本中有近一百万行（无注释）代码。图 6 表明，相对于系统的其他部分，该子系统一直在稳步增长，目前已占整个系统的 60% 以上。

驱动系统还是最多的

驱动子系统的规模和增长率实际上使得在图 5 中难以看出系统其余部分的情况。因此，图 7 展示了这些子系统相对于整个系统的占比。我们可以看到，arch、include、net 和 fs 子系统明显大于其余五个子系统。

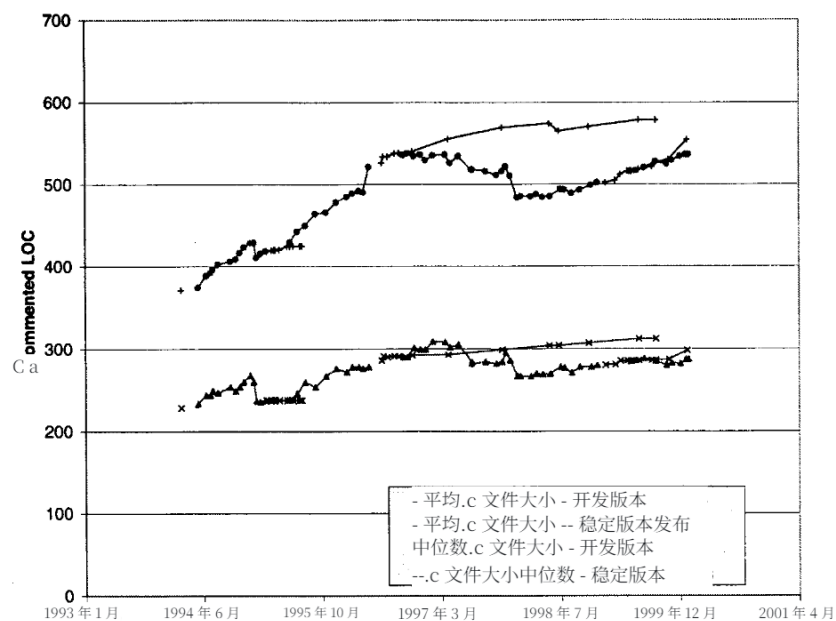


图 3. 实现文件（*.c”）的中位数大小和平均大小。

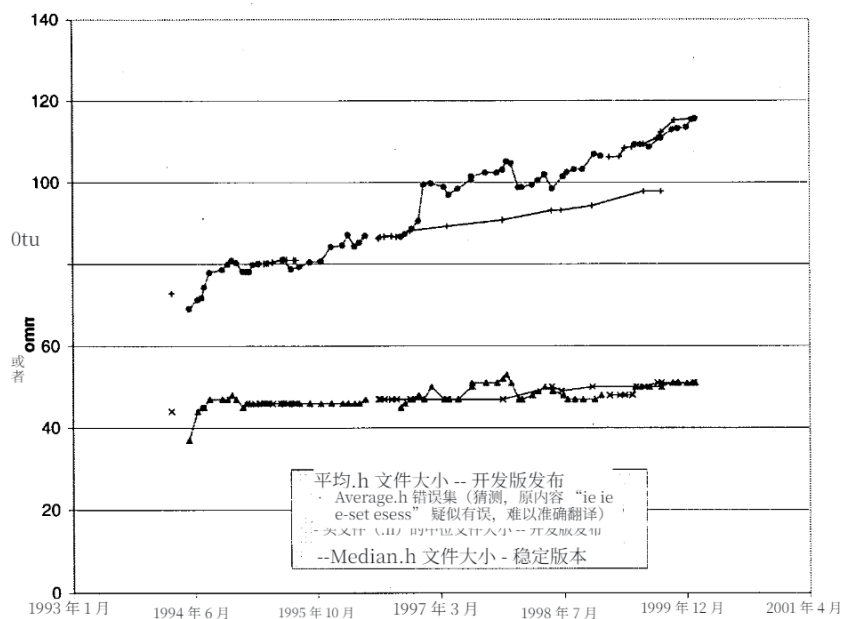


图 4. 头文件（*.h”）的中位数大小和平均大小。

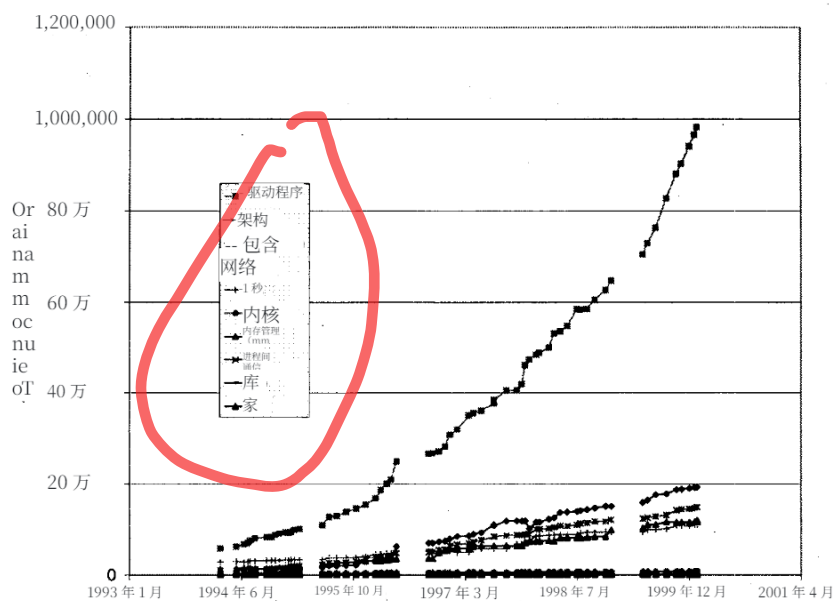


图 5. 主要子系统的增长情况（仅针对开发版本）。

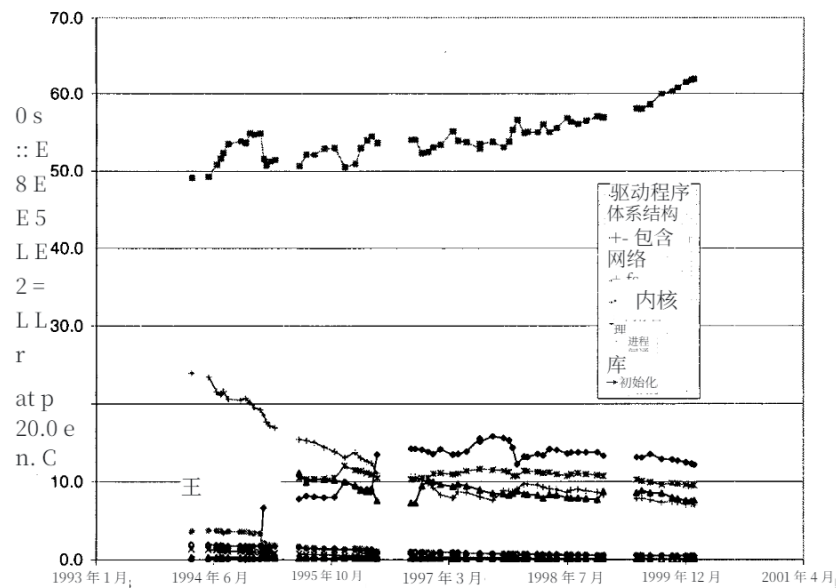


图 6. 各主要子系统占系统总代码行数的百分比（仅开发版本）。

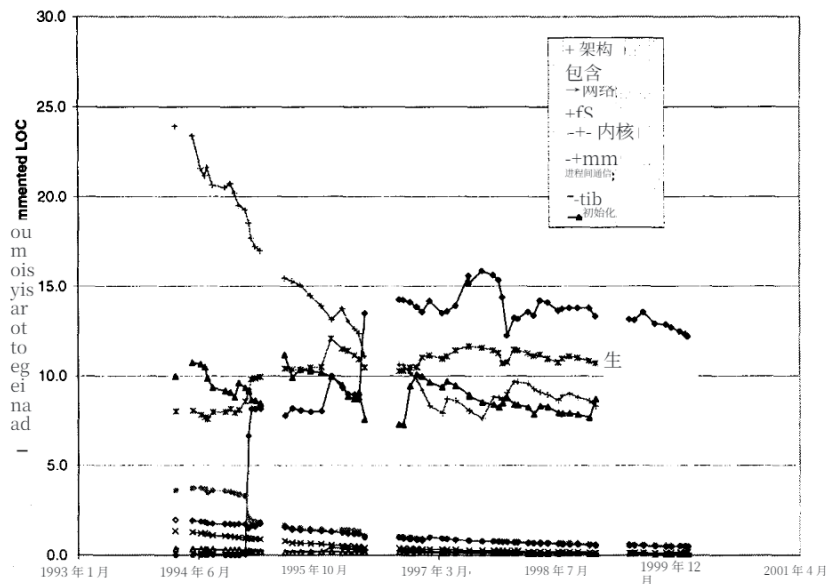


图 7. 各主要子系统占系统总代码行数的百分比（不包括驱动子系统，仅针对开发版本）。

并且这四个部分持续以看似线性或更好的速度增长。图 7 还表明，fs 子系统的增长速度比其他部分慢，并且随着时间的推移，其在整个内核大小中所占的相对比例显著下降。

图 8 展示了五个最小子系统的增长率。我们可以看到，虽然内核（kernel）和内存管理（mm）在稳步增长，但这些子系统实际上只包含很少量的代码。然而，这五个子系统是内核核心的一部分；无论目标硬件是什么，内核编译中几乎都会包含这些子系统的所有代码。此外，由于操作系统内核通常设计得尽可能小巧紧凑，这些核心子系统的过度增长可能会被视为一种不健康的迹象。

图 9 展示了驱动程序子系统开发版本的增长情况。增长规模最大且速度最快的是 drivers/net，它包含诸如以太网卡等网络设备的驱动程序代码。该子系统的增长反映了 Linux 所支持的网络设备数量、为这些设备创建驱动程序的相对复杂性，以及有时必须在源文件中存储大量特定于设备的数据这一事实。我们注意到，对于最新的开发内核（2.3.39），drivers 子系统中源文件的平均大小超过 600。

代码行数，这在主要子系统中平均数量最高。驱动程序的大多数子系统都呈现出显著增长，这表明随着越来越多的用户希望在多种不同品牌的设备上运行 Linux，Linux 的接受度日益提高。

然而，我们注意到驱动程序子系统的增长和规模扭曲了对 Linux 系统规模和复杂程度的认知。首先，设备驱动程序的本质是将常见且易于理解的请求转换为特定硬件设备能够高效执行的任务。设备驱动程序通常相当庞大和复杂，但也相对独立，彼此之间以及与系统的其他部分相互独立。其次，我们注意到虽然旧硬件可能会淘汰，但旧驱动程序往往会长期存在，以防某些用户仍有需求。因此，每个内核版本都会分发大量相对未使用（且相对缺乏维护）的“遗留”驱动程序。最后，我们注意到即使对于当前的设备驱动程序，大多数用户往往只需要可能的驱动程序中的少数几种。例如，两个最大的驱动程序子系统是网络（net）和小型计算机系统接口（scsi），然而如今绝大多数个人电脑在销售时都没有配备网卡或 SCSI 卡。

图 10 展示了 arch 子系统开发版本的增长情况，其中每个版本都代表着

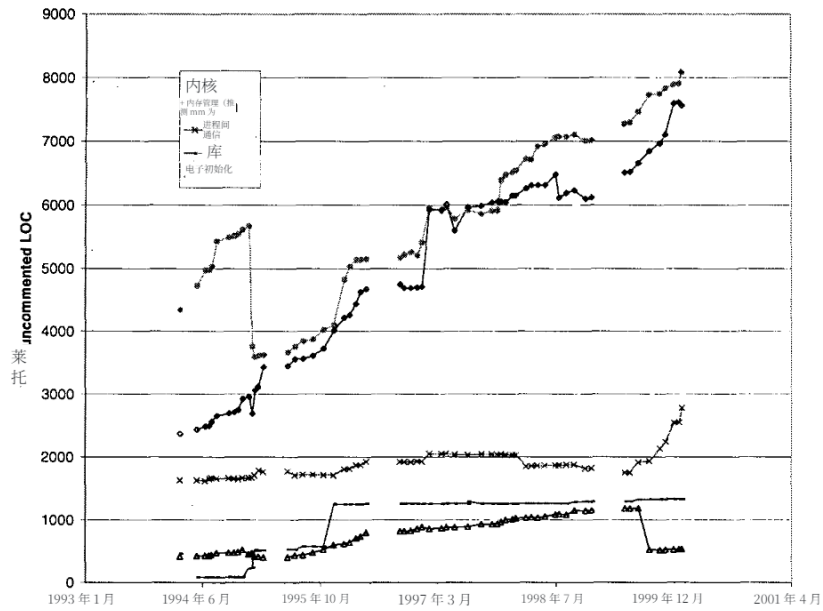


图 8. 规模较小的核心子系统的增长情况（仅限开发版本）。

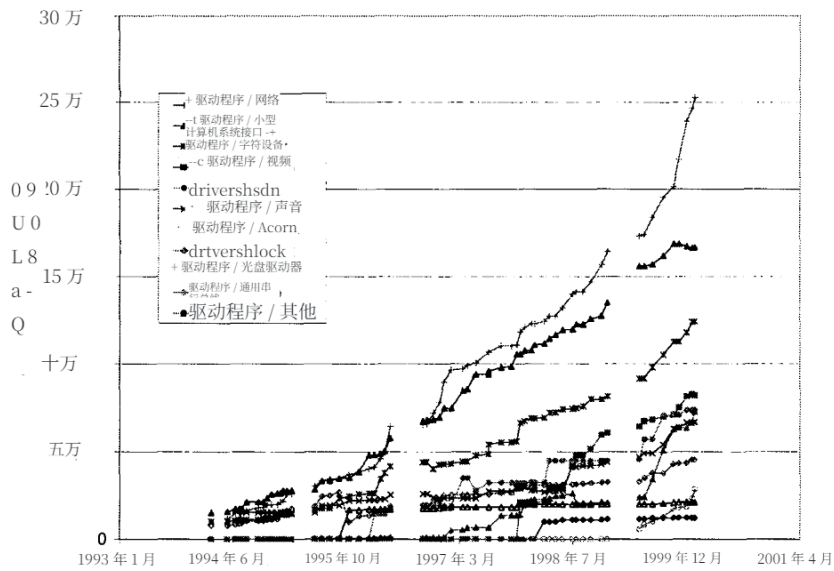


图 9. 驱动程序子系统的增长情况（仅开发版本）。

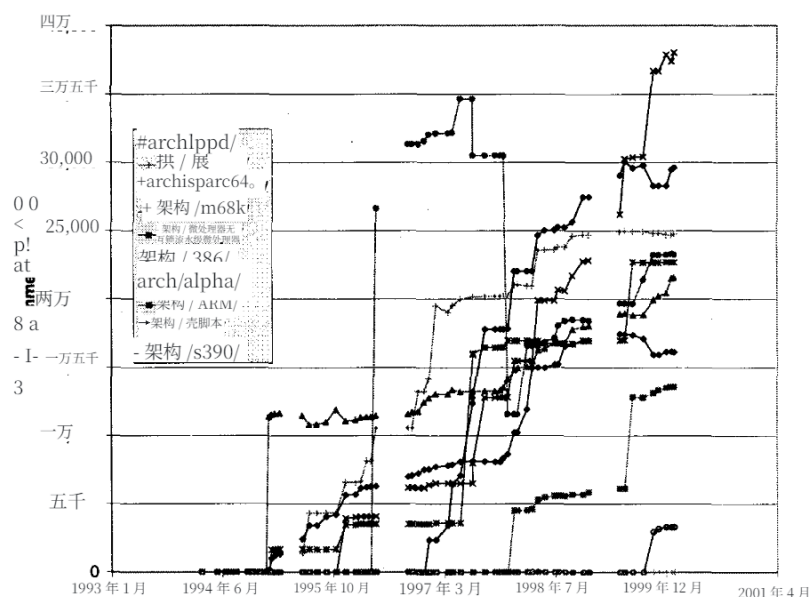


图 10. 架构子系统的增长情况（仅适用于开发版本）。

代表了一种由 Linux 支持的主要 C/PW 硬件架构。这里最有意思的是许多这类子系统实现的突然跃升。第一次这样的跃升发生在 1995 年初，当时决定将对 Alpha、Sparc 和 Mips 架构的支持整合到 Linux 中，这也导致专门针对英特尔 386 架构的代码从主内核子系统转移到 arch/i386。其他 arch 子系统随后的跃升，是由于一次性添加大量外部开发的代码。在许多情况下，这种架构支持由相对独立的开发团队（有时是公司）独立于 Linux 主要开发内核版本进行开发和维护。这类子系统常见的发展模式是，在添加重大新修订时出现大幅跃升，随后是相对稳定的时期，期间只涉及小修订和错误修复。

7 结论

Linux 操作系统内核是一个广泛使用的大型软件系统的非常成功的例子，它是使用“**开源**”开发（OSD）模型开发的。我们使用了几种指标研究了 Linux 在其六年生命周期中的发展情况，并且发现，从系统层面来看，它的发展呈现超线性。考虑到以下因素，这种强劲的增长率似乎令人惊讶：(a)

其一，它的规模庞大（包括注释和空行在内，代码行数超过两百万行）；其二，它的开发模式（由高度协作且分布在不同地理位置的开发者组成，其中许多人免费贡献自己的时间和精力）；其三，先前发表的研究表明，大型软件系统的增长往往会随着系统规模的增大而放缓 [14.4.23]。

我们发现，正如 Gall 等人 [4] 先前所指出的，研究子系统的增长模式，有助于更好地理解该系统为何以及如何能够如此成功地进化。我们进一步认为，“黑箱”式的研究是不够的；必须探究子系统的本质，研究它们的进化模式，才能理解整个系统为何以及如何进化。我们发现，虽然 Linux 的整个源代码树非常庞大，但一半以上的代码是由彼此相对独立的设备驱动程序组成；我们还发现，其余系统的很大一部分是特定于某些 CPU 的并行特性；而小型的核心内核子系统仅占整个源代码树的一小部分。也就是说，Linux 操作系统内核并不像看上去那么庞大，因为（根据我们自己的实验）任何编译版本可能只包含整个源代码树中 15% 到 50% 的源文件。

最后，我们认为这个案例研究很重要

大型软件系统演化研究中的数据点。我们希望这将鼓励对开源软件开发 (OSD) 软件系统的演化进行进一步研究, 并与使用更传统方法开发的系统进行比较。

8 致谢

我们感谢大卫·托曼 (David Toman) 对 FreeBSD 操作系统提出的一些有益意见, 也感谢休·奇普曼 (Hugh Chipman) 和戴尔·舒尔曼斯 (Dale Schuurmans) 在统计建模方面提供的帮助。

参考文献

- I. T. 鲍曼和 R. C. 霍尔特。重构所有权架构以助理解软件系统。发表于 1999 年 5 月在宾夕法尼亚州匹兹堡举行的 1999 年 IEEE 程序理解研讨会 (IWPC' 99) 论文集。
- I. T. 鲍曼、R. C. 霍尔特和 N. V. 布鲁斯特。以 Linux 为例: 其提炼出的软件架构。发表于 1999 年 5 月在加利福尼亚州洛杉矶举行的第 21 届巴西软件工程大会 (ICSE - 21) 论文集。
- S. G. 艾克、T. L. 格雷夫斯、A. E. 卡尔、J. S. 马龙和 A. 莫库。代码会衰退吗? 从变更管理数据评估证据。即将发表于《IEEE 软件工程汇刊》。
- H. 高尔、M. 贾扎耶里、R. 克洛施和 G. 特劳斯穆斯。基于产品发布历史的软件演化观察。载于 1997 年国际软件维护大会 (ICSM' 97) 会议录, 意大利巴里, 1997 年 10 月。
- D. 希伯特。Exuberant CTags 主页。网站。<http://home.HiWAAY.neVdarren/ctags/>。
- <http://www.freebsd.org>。FreeBSD 主页。网站。
- <http://www.kernelnotes.org>。kernelnotes.org: Linux 内核信息官方网站。网站。
- <http://www.kernel.org>。Linux 内核档案库。网站。
- <http://www.linux.org>。Linux 主页。网站。
- <http://www.opensource.org>。开源项目主页。网站。
- <http://www.vim.org>。VIM 主页。网站。
- C. E. 凯默勒和 S. 斯劳特。一种研究软件演化的实证方法。《IEEE 软件工程汇刊》, 第 25 卷第 4 期, 1999 年 7/8 月。
- M. M. 莱曼和 L. A. 贝拉迪。《程序演化: 软件变化过程》。学术出版社, 1985 年。
- M. M. 莱曼、D. E. 佩里和 J. E. 拉米儿。演化指标对软件维护的影响。发表于 1998 年国际软件维护大会 (ICSM' 98) 论文集, 马里兰州贝塞斯达, 1998 年 11 月。
- M. M. 莱曼、J. E. 拉米儿、P. D. 韦米克、D. E. 佩里和 W. M. 图尔斯基。《软件演化的度量与定律——九十年代的观点》, 载于第四届国际软件度量研讨会 (Metrics' 97) 论文集, 新墨西哥州阿尔伯克基, 1997 年。
- D. L. 帕马斯。《软件老化》, 收录于 1994 年 5 月在意大利索伦托举行的第 16 届国际软件工程大会 (ICSE - 16) 论文集。

D. E. 佩里。软件演化的维度。收录于 1994 年国际软件维护大会 (ICSM' 94) 会议论文集, 1994 年。

E. S. 雷蒙德。《大教堂与集市: 一个偶然革命者对 Linux 和开源的思考》。奥莱利联合公司, 1999 年 10 月。

A. R. 网站。<http://www.linuxhq.com/guides/TLIUtlk.html>。

C. 萨尔岑伯格。黄玉: 面向 22 世纪的 Perl》。<http://www.perl.com/pub/1999/09/topaz.html>。

J. B. 陈、M. W. 戈弗雷、E. H. S. 李和 R. C. 霍尔特。开源软件的架构分析与修复。收录于《2000 年国际程序理解研讨会 (IWPC' 00) 论文集》, 爱尔兰利默里克, 2000 年 6 月。

J. B. 陈和 R. C. 霍尔特。软件架构的正向与反向修复。收录于 1999 年 11 月在多伦多举办的 1999 年加拿大高级计算机科学会议论文集。

W. M. 图尔斯基。软件系统平稳增长的参考模型。《IEEE 软件工程汇刊》, 1996 年 8 月, 第 22 卷第 8 期。

L. 韦普斯塔斯。IBM ESA/390 大型机架构上的 Linux 系统。网站。<http://linas.org/linux/i370/i370.html>。